

情報科学基礎論 推論とプログラム 課題レポート

氏名：QIN YUANZHI
学籍番号：6930-38-5717

提出日：2026 年 5 月 21 日

1 開発したプログラムの概要

本課題では、生成 AI である Codex を活用し、Python によるコマンドライン ToDo リストアプリケーションを開発した。プログラム名は `todo.py` であり、ユーザーはメニューを選択することで、タスクの追加、一覧表示、完了状態への変更、削除を行うことができる。

また、タスク情報は `tasks.json` に JSON 形式で保存される。そのため、プログラムを終了して再起動しても、前回登録したタスクを継続して利用できる。さらに、空のタスク名、範囲外の番号、数値でない入力などに対する入力検証を実装し、JSON ファイルの破損や読み書きエラーに対する例外処理も追加した。

機能	内容
タスク追加	ユーザーが入力した内容を新しいタスクとして登録する
タスク表示	登録済みタスクを番号付きで表示し、完了・未完了の状態も示す
完了登録	指定した番号のタスクを完了済みに変更する
タスク削除	指定した番号のタスクを一覧から削除する
永続化	タスクを <code>tasks.json</code> に保存し、起動時に自動で読み込む
入力検証	空入力、不正なメニュー番号、不正なタスク番号を処理する
例外処理	JSON の形式不正、ファイル読み書きエラー、入力中断に対応する

2 使用した生成 AI と開発環境

本開発では、生成 AI として Codex を利用した。Codex には、プログラムの生成、機能追加、入力検証、JSON による永続化、例外処理、単体テストの作成、README および本レポートの作成を依頼した。

項目	内容
生成 AI	Codex
プログラミング言語	Python 3
保存形式	JSON
テスト方法	Python 標準ライブラリ <code>unittest</code>
主なファイル	<code>todo.py</code> , <code>tasks.json</code> , <code>test_todo.py</code> , <code>README.md</code>

外部ライブラリは使用せず、Python 標準ライブラリのみで実装した。これにより、環境依存性を小さくし、提出後にも比較的簡単に実行できるようにした。

3 開発過程

3.1 初期実装

最初に、メニュー形式の ToDo リストアプリケーションを作成した。初期版では、タスクの追加、表示、完了登録、削除、終了の 5 つの操作を実装した。タスクはプログラム実行中のリストとして保持され、各タスクは `title` と `completed` を持つ辞書として表現した。

この段階では、プログラム終了後にデータが消えるという課題があった。そのため、次の段階で永続化を追加した。

3.2 JSON による永続化

次に、タスクを `tasks.json` に保存する機能を追加した。プログラム起動時には `load_tasks()` により JSON ファイルを読み込み、タスク追加・完了登録・削除のたびに `save_tasks()` を呼び出して保存する構成にした。

`tasks.json` が存在しない場合は、空のリスト `[]` から開始するようにした。これにより、初回実行時にもエラーが発生せず、自然にプログラムを利用できる。

3.3 入力検証の追加

その後、入力検証を改善した。具体的には、空のタスク説明を拒否し、メニュー選択で空入力や数値以外の入力、1 から 5 の範囲外の値を処理した。また、タスク番号についても、空入力、文字列、0、範囲外の番号を受け付けないようにした。

入力検証を分離するため、メニュー選択には `get_menu_choice()`、タスク番号選択には `get_task_number()` を用いた。これにより、同じような検証処理が複数箇所に散らばることを避け、読みやすい構成にした。

3.4 例外処理の追加

さらに、JSON ファイルに関する例外処理を追加した。`tasks.json` の内容が不正な JSON である場合、読み込み時に `json.JSONDecodeError` が発生する。この場合は警告を表示し、空のタスクリストで開始するようにした。

また、読み書き時の `OSError` にも対応し、ファイルを読み込めない場合や保存できない場合でも、プログラム全体が突然終了しないようにした。さらに、ユーザーが `Ctrl+C` を押した場合や入力が予期せず終了した場合も、`KeyboardInterrupt` と `EOFError` を捕捉して終了メッセージを表示するようにした。

3.5 単体テストの作成

最後に、Python の標準ライブラリである `unittest` を用いて単体テストを作成した。テストファイルは `test_todo.py` である。

テストでは、実際の `tasks.json` を汚さないように、一時ディレクトリ内の仮の JSON ファイルを使用した。また、`unittest.mock.patch` を用いてユーザー入力や標準出力を差し替え、手動操作なしで自動的にテストできるようにした。

4 開発モデルの観点からの考察

4.1 ウォーターフォールモデル

ウォーターフォールモデルでは、要求定義、設計、実装、テスト、保守という工程を順番に進める。本開発でも、最初に「ToDo リストとして必要な機能」を整理し、その後に実装、テスト、文書化へ進んだ点では、ウォーターフォール的な側面がある。

ただし、実際には最初から完全な仕様が決まっていたわけではない。途中で永続化、入力検証、例外処理、単体テストを順番に追加していったため、純粋なウォーターフォールモデルとは異なる。

4.2 スパイラルモデル

スパイラルモデルでは、開発を小さなサイクルに分け、各サイクルでリスク分析、実装、評価を繰り返す。本開発はこの考え方に近い。最初に基本機能を作り、次に「データが消える」という問題を解決するため JSON 保存を追加し、その後「誤入力で不自然な動作になる」というリスクに対して入力検証を追加した。

さらに、JSON ファイル破損や読み書き失敗というリスクに対して例外処理を追加し、最後に単体テストで確認した。このように、問題点を見つけるたびに改善する流れは、スパイラルモデルの反復的な開発に近い。

4.3 アジャイル開発

アジャイル開発では、小さく動くものを早く作り、フィードバックを受けながら改善を続ける。本開発では、Codex に対して段階的に要求を与え、各段階でプログラムを実行・確認しながら改善した。これはアジャイル的な開発方法である。

特に、最初に最小限の ToDo リストを作り、その後に永続化、入力検証、例外処理、単体テストを追加した点は、機能を小さな単位で増やすアジャイル開発に近い。生成 AI を利用すると、短いサイクルで実装と確認を繰り返しやすかった。

5 Verification と Validation

5.1 Verification の観点

Verification は「作られたものが仕様通りに正しく実装されているか」を確認する活動である。本プログラムでは、以下の方法で Verification を行った。

第一に、単体テストを作成した。`test_todo.py` には 11 個のテストがあり、主に次の内容を確認している。

テスト対象	確認内容
<code>load_tasks()</code>	<code>tasks.json</code> が存在しない場合に空リストを返す
<code>save_tasks()</code> / <code>load_tasks()</code>	保存したタスクを正しく読み込める
不正な JSON	JSON が壊れていても空リストで開始する
不正なデータ形式	リスト以外の JSON や不正なタスク形式を拒否する
<code>add_task()</code>	空の説明を拒否し、有効な説明を追加する
<code>get_menu_choice()</code>	不正なメニュー入力を拒否し、有効な入力を受け付ける
<code>get_task_number()</code>	不正なタスク番号を拒否する
<code>mark_completed()</code>	指定タスクを完了済みにする

`delete_task()`

指定タスクを削除する

テストは以下のコマンドで実行できる。

```
python3 -m unittest test_todo.py
```

実行結果は次の通りである。

```
Ran 11 tests  
OK
```

第二に、構文エラーがないことを確認するため、次のコマンドも実行した。

```
python3 -m py_compile todo.py test_todo.py
```

このように、単体テストと構文確認により、プログラムが仕様に沿って動作することを検証した。

5.2 Validation の観点

Validation は「作られたものが利用者の目的に合っているか」を確認する活動である。本課題で求められる目的は、コマンドライン上で簡単な ToDo リストを操作できることである。

実際にプログラムを実行すると、ユーザーはメニューから操作を選択できる。例えば、タスクを追加し、一覧表示で確認し、完了済みに変更し、不要になったタスクを削除できる。この一連の操作は、ToDo リストアアプリケーションとして期待される基本的な利用場面を満たしている。

また、入力ミスをしてプログラムがすぐに終了せず、適切なエラーメッセージを表示して再入力できるため、利用者にとって扱いやすい。さらに、タスクが JSON ファイルに保存されるため、日常的に使う ToDo リストとして最低限必要な継続性も備えている。

以上より、本プログラムは単にコードとして正しいだけでなく、利用者が ToDo リストとして使うという目的にも合っていると判断できる。

6 生成 AI を用いた V&V の有用性

生成 AI を用いることで、V&V の作業を効率的に進めることができた。特に、単体テストの作成では、どの関数をどのようにテストすべきかを Codex が提案し、`unittest` と `mock` を用いたテストコードを短時間で作成できた。

また、生成 AI は人間が見落としやすい例外処理の観点を補助できる。たとえば、通常の利用では `tasks.json` が壊れている場合を考えないことが多い。しかし Codex を利用することで、不正な JSON、ファイル読み書き失敗、入力中断などのケースを検討し、プログラムをより頑健にできた。

さらに、テスト実行結果を確認しながら、不要な出力を抑えるなどの細かい改善も行えた。生成 AI は単にコードを書く道具ではなく、検証すべき観点を広げる補助者としても有用であると感じた。

7 生成 AI を用いた V&V の問題点

一方で、生成 AI には注意すべき問題点もある。第一に、生成 AI が作成したテストが、本当に要求を十分に確認しているとは限らない。テストが通っても、利用者の目的に合っているかどうかは人間が確認する必要がある。

第二に、生成 AI はもっともらしいコードや説明を出力するが、その内容が常に正しいとは限らない。特に例外処理やファイル操作では、実際に実行して確認しなければ問題に気づけない場合がある。そのため、生成されたコードをそのまま信頼するのではなく、テスト実行や手動確認を行う必要がある。

第三に、生成 AI によって開発速度が上がる一方で、開発者自身がコードの意味を理解しないまま進めてしまう危険がある。V&V の目的は、単にエラーをなくすことではなく、仕様と利用目的に合っていることを確認することである。そのため、生成 AI の出力を読み、必要に応じて修正できる理解が重要である。

8 まとめ

本課題では、Codex を用いて Python のコマンドライン ToDo リストアプリケーションを開発した。プログラムは、タスクの追加、表示、完了登録、削除、JSON による保存と読み込み、入力検証、例外処理を備えている。

開発過程は、最初に基本機能を作り、実行しながら問題点を見つけて改善する形で進めた。そのため、ウォーターフォールモデルの工程的な整理を含みつつ、スパイラルモデルやアジャイル開発に近い反復的な進め方となった。

Verification としては、単体テストと構文チェックを行い、プログラムが仕様通りに動作することを確認した。Validation としては、実際のメニュー操作を通じて、ToDo リストとして利用者の目的を満たしていることを確認した。

生成 AI は、実装だけでなくテスト作成や例外処理の検討にも有用であった。しかし、生成された内容をそのまま信頼するのではなく、人間が要求、実行結果、テスト内容を確認する必要がある。したがって、生成 AI は開発者の代わりになるものではなく、開発と V&V を支援する道具として利用するのが適切である。

参考・補足

本レポートでは、他者のコードや文章の直接引用は使用していない。ソースコードは `todo.py`、単体テストは `test_todo.py`、保存データは `tasks.json` として、レポート PDF と同時に提出する。